



# UNDERSTANDING AND IMPLEMENTING KUBERNETES HORIZONTAL POD AUTOSCALER (HPA)

## Technical Note

Date: 7 January 2025

Version: 1.0

This document contains copyrighted work and proprietary information belonging to K2view. All intellectual property rights (including, without limitation, copyrights, trade secrets, trademarks, etc.) evidenced by or embodied in and/or attached, connected, or related to this document, as well as any information contained herein, are and shall be owned solely by K2view. K2view does not convey to you an interest in or to this document, the information contained herein, or its intellectual property rights, but only a personal, limited, fully revocable right to use the document solely for review purposes. Unless explicitly set forth otherwise, you may not reproduce by any means any document and/or copyright contained herein.

Information in this document is subject to change without notice. Corporate and individual names and data used in the examples herein are fictitious unless otherwise noted.

Copyright © 2025 K2view Ltd. / K2VIEW LLC. All rights reserved.

The following are trademarks of K2view: K2view logo, K2view's platform.

K2view reserves the right to update this list document from time to time.

## Table of Contents

|                                                 |   |
|-------------------------------------------------|---|
| Introduction.....                               | 3 |
| How HPA Works .....                             | 3 |
| Key Considerations for HPA.....                 | 4 |
| Recommendations .....                           | 5 |
| Is HPA Right for Your Workload? .....           | 5 |
| Test HPA's Suitability for Your Use Cases ..... | 5 |
| Conclusion .....                                | 5 |
| Sample HPA Configuration .....                  | 6 |

## Introduction

The **Horizontal Pod Autoscaler (HPA)** in Kubernetes is a dynamic tool designed to automatically scale the number of pods in a workload based on observed metrics such as CPU, memory, or custom application-defined metrics. While it offers significant operational efficiencies, its suitability depends on the specific workload characteristics and use case requirements.

This technical note provides an overview of HPA, including its working mechanism, a sample configuration, advantages and disadvantages, key considerations, and recommendations for implementation.

We encourage the reader to review the Horizontal Pod Autoscaling documentation offered by the Kubernetes community at <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>.

## How HPA Works

HPA dynamically adjusts the number of pods in a Kubernetes workload to maintain desired performance levels and optimize resource utilization. Here's an overview of its functionality:

1. **Metrics Collection:** HPA fetches resource utilization data from the Kubernetes metrics server (e.g., CPU and memory).
2. **Scaling Decision:** The collected metrics are compared against target thresholds defined in the HPA configuration. If resource usage exceeds or falls below the thresholds, HPA calculates the desired number of pods required to restore balance.
3. **Adjustment:** Pods are scaled up or down to match the calculated requirements. The scaling process is periodic and adheres to constraints such as minimum and maximum replica counts.

You can benefit from HPA by leveraging its ability to offer:

- **Operational Efficiency:** Automates scaling decisions, reducing manual intervention.
- **Cost Optimization:** Matches resource allocation with demand, minimizing over-provisioning.

## Key Considerations for HPA

If everything works as expected and the HPA is well suited to your use cases, it will provide an excellent solution for adapting to workload needs, but you need to consider its use carefully. Here are key considerations:

1. **Workload Suitability:** Determine if the workload's performance and demand patterns align with HPA's capabilities. For example, web applications with predictable traffic patterns are well-suited for HPA.
2. **Scaling Boundaries:** Set realistic minReplicas and maxReplicas values to balance performance and resource efficiency.
3. **Testing and Monitoring:** Before deployment, test HPA configurations in a staging environment and monitor scaling behavior, resource utilization, and node distribution to identify potential issues.

It does come with some challenges, namely:

- **Metrics Dependency:** Requires reliable data from the Kubernetes metrics server or external metrics providers. Default configurations rely on CPU and memory, which may not suit all workloads.
- **Scaling Lag:** Decisions are made periodically, introducing delays in responding to traffic surges. The system may not react quickly to sudden traffic spikes.
- **Resource Imbalance:** This can lead to uneven node utilization or inefficient scaling in complex scenarios.
- **Complexity:** Misconfigured HPA settings can lead to instability, such as oscillating pod counts or under-provisioning.

Let's look at a few examples.

In scenarios where you have, for instance, three nodes, each with a CPU limit of 2, and the scaling threshold is set at 80%, problems can arise from uneven load distribution. If one node handles a task requiring 6 cores (due to an intensive or long-running job), the HPA will calculate the average load across all three nodes. This can push the average above the threshold, triggering a scale-up, even though only one node is overloaded while the others remain mainly idle. This leads to inefficient resource usage.

Another inefficiency occurs during the scaling-down process. After scaling up, the HPA waits for a cooldown period (e.g., a `stabilizationWindowSeconds` of 300) before initiating a scale-down. During this interval, jobs may be scheduled on the newly added nodes. However, when the scale-down process starts, it might overlook the workload on specific pods and terminate them, disrupting running tasks. This behavior can negatively impact overall system efficiency.

HPA's auto-scaling isn't universally efficient and needs to be assessed case-by-case. While it may perform well for some projects (e.g., stateless API request processing), it could lead to more problems than benefits in others. This is why we do not enable HPA by default; it requires careful consideration and thorough evaluation of its suitability.

## Recommendations

Kubernetes offers detailed documentation on HPA algorithms, how scaling is calculated, and examples of its operation to aid decision-making. Please refer to <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. These resources are essential for understanding HPA's behavior and determining its appropriateness for a specific use case.

### Is HPA Right for Your Workload?

HPA is a powerful tool, but it is not a one-size-fits-all solution. Careful consideration is essential to determine its suitability for your use case:

- **Evaluate Workload Patterns:** Ensure HPA can handle the scaling requirements of your workload, including traffic spikes or high variability.
- **Test Configurations:** Validate the behavior of scaling policies under realistic scenarios in a controlled environment.
- **Monitor for Inefficiencies:** Watch for resource imbalance, scaling delays, or suboptimal node utilization.

### Test HPA's Suitability for Your Use Cases

If you are considering HPA for Fabric StatefulSet, please test its use for your use cases. For example:

- Use the [Sample HPA Configuration](#) with minReplicas: 3 and maxReplicas: 5.
- Start with CPU-based scaling (targetCPU Utilization Percentage: 90).
- Closely monitor resource usage and scaling events to fine-tune configurations.

## Conclusion

When adequately employed, HPA can significantly enhance workload performance and resource efficiency. However, its effectiveness depends on understanding the workload requirements, configuring metrics accurately, and testing behavior under real-world conditions.

Organizations should approach HPA with a customized strategy that leverages Kubernetes' inherent flexibility to maximize benefits while minimizing risks. Adopting HPA with thorough testing and monitoring guarantees scalability and cost efficiency without sacrificing system stability.

## Sample HPA Configuration

Please refer to [https://github.com/k2view/blueprints/blob/main/helm/fabric/templates/hpa\\_autoscaling.yaml](https://github.com/k2view/blueprints/blob/main/helm/fabric/templates/hpa_autoscaling.yaml) Helm definition and its associated <https://github.com/k2view/blueprints/blob/main/helm/fabric/values.yaml> values definition.

Simplified, here is a sample HPA configuration:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: example-hpa
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: example-deployment
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

### Explanation:

- `scaleTargetRef`: Specifies the workload (e.g., Deployment, StatefulSet) being scaled.
- `minReplicas` **and** `maxReplicas`: Define the bounds for scaling.
- `metrics`: Include resource metrics such as CPU and memory with target utilization thresholds.